

Part 1: The Phantom Bug

- Write a program that tosses 50,000,000 random (x, y) points into a 1×1 square to approximate π ($\pi \approx 4 \times \frac{\text{hits}}{\text{total}}$).
- Split the loop across 4 native Java threads.
- Have all 4 threads update a shared variable: `static long totalHits = 0;` via `totalHits++`.
- **Deliverable:** Run it 5 times. Record the value of π .
- *The Lesson:* The code runs fast, but gives $\pi \approx 1.8$ to 2.4 because threads overwrite each other's memory writes (data race).

Part 2: The Synchronization Trap

- Fix the race condition by adding `synchronized` to the increment or using `AtomicLong.incrementAndGet()`.
- **Deliverable:** Measure execution time vs. a plain, single-threaded for loop.
- *The Lesson:* The answer is now accurate (≈ 3.1415), but it runs drastically slower than a single thread because cores spend 95% of their cycles stalling on memory bus locks.

Part 3: OpenMP-Style Reduction

- Eliminate shared-state locks. Give each thread a **private, local counter** (conceptually identical to `#pragma omp parallel for reduction(+:totalHits)`).
- Accumulate local hits inside the thread, then add the partial sums together *only once* at the very end when joining threads.
- **Deliverable:** Benchmark total execution time for $T \in \{1, 2, 4, 8, 16, 32\}$ threads on their machine with 100,000,000 iterations.

- Fill out this grid:

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread (TN T1)	Efficiency (TSpeedup)
1	⁴⁶⁰⁹ (baseline)	1.0x	100%
2	²⁵⁴⁷ ...	^{1.18} ...	^{90.5} ...
4	¹⁷⁰⁴ ...	^{2.70} ...	^{67.6} ...
8	⁹⁹¹ ...	^{4.65} ...	^{58.1} ...
16	⁸¹⁶ ...	^{5.65} ...	^{35.3} ...

Questions

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?
2. Why was the synchronized version in Part 2 slower than running on one single core?

Upload your solutions before the class end-time to the Git repo that you created for benchmarks. Share repo access with @sufyanism (GitHub/GitLab).

1. My CPU only has 8 real cores, so at 8 threads I'm already using all of them. Going to 16 threads means 2 threads now share each core, so they take turns instead of running at the same time. I still get a little boost (4.65x → 5.65x) probably from hyperthreading letting a core do useful work while the other thread is waiting on memory, but it's nowhere near double — and my efficiency dropped from 58% to 35%, showing the extra threads are mostly just overhead now.

2. Because locking makes the threads take turns instead of actually running at the same time — only one thread can touch the counter at once, so the others just wait. Plus every lock/atomic update has to bounce the shared value between CPU cores' caches, which is really slow. So 4 threads fighting over one variable end up slower than 1 thread that never has to share or wait at all.